Programação em Python

senac
tecno
logia

Desenvolver Aplicações Front-End para Web

senac
tecno
logia

Bem-vindos à Aula:
Desenvolver **Aplicaçoes Front–End para web**

O que faremos hoje:

- Wireframe

Semantix

Situação de **Aprendizagem**

Senac tecnologia

Site Profissional (JWC)

Para desenvolver uma aplicação para um dos **clientes da JWC**

Missão

Criar um site profissional que represente a marca, gere credibilidade e ofereça uma experiência incrível ao usuário.

Cliente

JWC – Tecnologia e Soluções Digitais

JWC

Inovação. Qualidade. Resultados.

Tema Livre

Site institucional para apresentar serviços, portfólio e formas de contato.

Modelo de Projeto

/projeto-site

- projeto-site/
  - index.html
  - contato.html
  - css/
    - style.css
  - js/
    - script.js
  - img/
    - logo-jwc.png
    - banner.jpg
    - servico1.jpg
    - servico2.jpg
    - cliente.jpg

Estrutura simples, organizada e pronta para desenvolvimento profissional e escalável.

Tecnologias Utilizadas

- **HTML5** – Estrutura
- **CSS3** – Estilização
- **JavaScript** – Interatividade
- **Design** – Responsivo
- **Performance** – Otimizada
- **Acessibilidade** – WCAG
- **Git** – Versionamento

Semantix

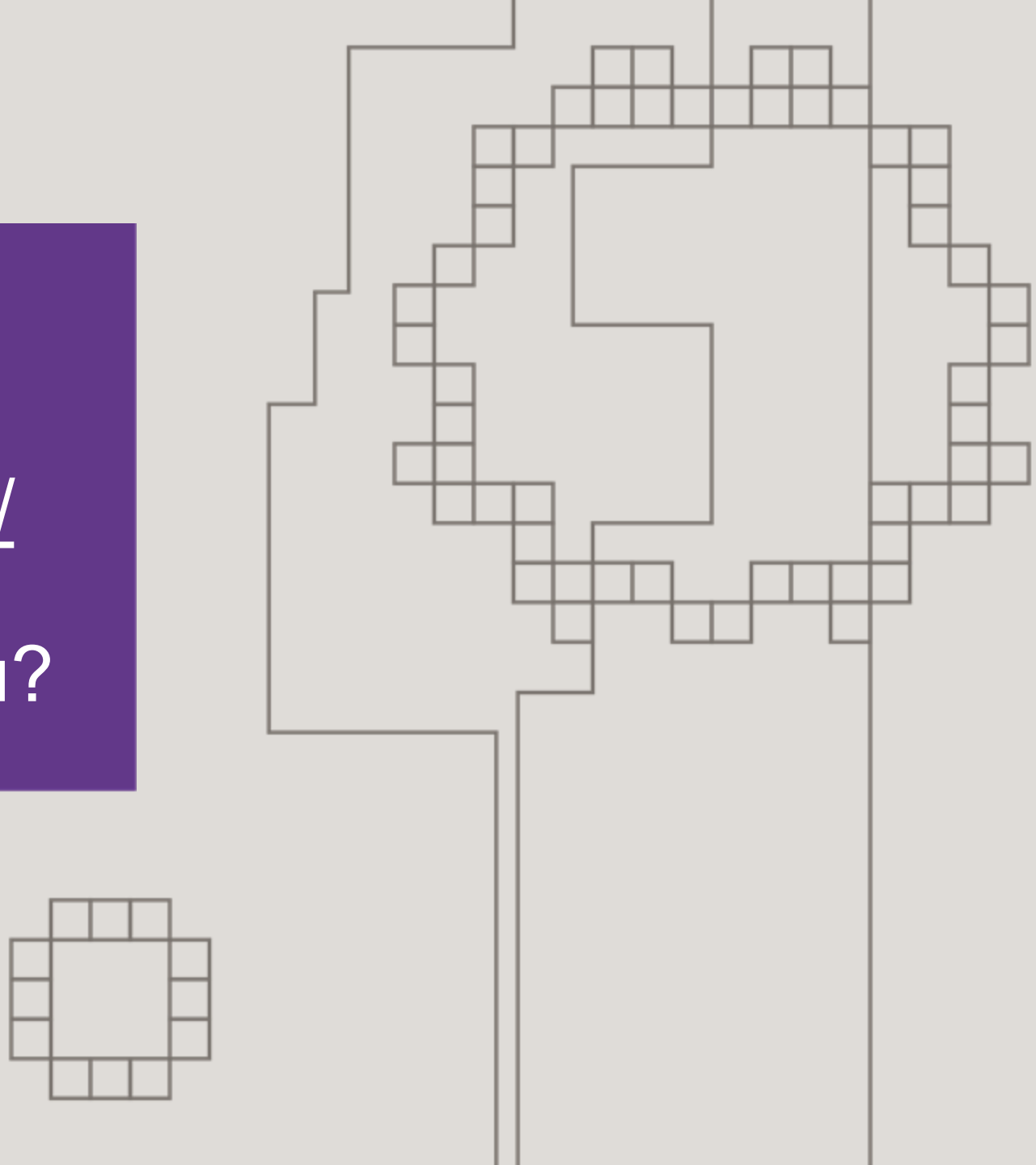
Sites Bons vs. Ruins

Senac tecnologia

Acesse:

https://www.arngren.net/

e responda o que achou?



Acesse:

https://www.gov.br/pt-br

e responda o que achou?

O que torna um site profissional?

Semantix

O que é Wireframe?

Senac tecnologia

Imagine que você vai construir um prédio. Você não começa pintando as paredes de azul e escolhendo as cortinas. Primeiro, você faz o **desenho técnico** para saber onde ficam as vigas, as portas e as janelas. **O Wireframe é esse desenho.**

1. Comece pelo objetivo

Antes de desenhar qualquer coisa, responda:

- Qual é o objetivo da página?
- O que o usuário precisa fazer aqui?

Exemplo:
Home de loja → destacar produtos + levar para compra.

2. Pense na jornada do usuário

Um bom wireframe não é só layout — é fluxo.

Pergunte:

- O que o usuário vê primeiro?
- Para onde ele vai depois?

👉 Regra de ouro: **hierarquia visual clara**

3. Use estrutura simples (low fidelity primeiro)
Evite cores, imagens reais e detalhes no início.

Use:

- Caixas (divs)
- Linhas
- Textos genéricos (Lorem ipsum)
- Placeholder de imagem

Isso ajuda a focar no **layout**, não no visual

4. Mobile first (ESSENCIAL hoje)

Sempre pense primeiro no celular:

- Layout em coluna única
- Botões grandes
- Espaçamento confortável

Depois adapte para desktop

5. Use padrões de mercado (UX)

Não invente tudo do zero. Use padrões conhecidos:

- Header no topo
- Menu hambúrguer no mobile
- Banner (hero)
- Cards de produto
- Rodapé com contatos

Isso facilita a usabilidade

6. Trabalhe com grids e alinhamento

Um wireframe profissional é organizado:

- Use grid (colunas)
- Alinhe elementos
- Mantenha espaçamentos consistentes

Evita aquele layout “bagunçado”

7. Defina hierarquia de conteúdo

Use tamanhos diferentes para mostrar importância:

- Título grande
- Subtítulo médio
- Texto menor

O olho do usuário precisa “saber para onde olhar”

8. Destaque ações importantes

Todo wireframe precisa de ação:

- Botões visíveis
- Texto claro (ex: “Comprar”, “Ver produtos”)

Não esconda o objetivo da página

9. Pense em repetição (componentes)
Exemplo:

- Cards de produto iguais
- Botões padronizados

Isso facilita:

- Desenvolvimento
- Consistência visual

10. Teste antes de finalizar

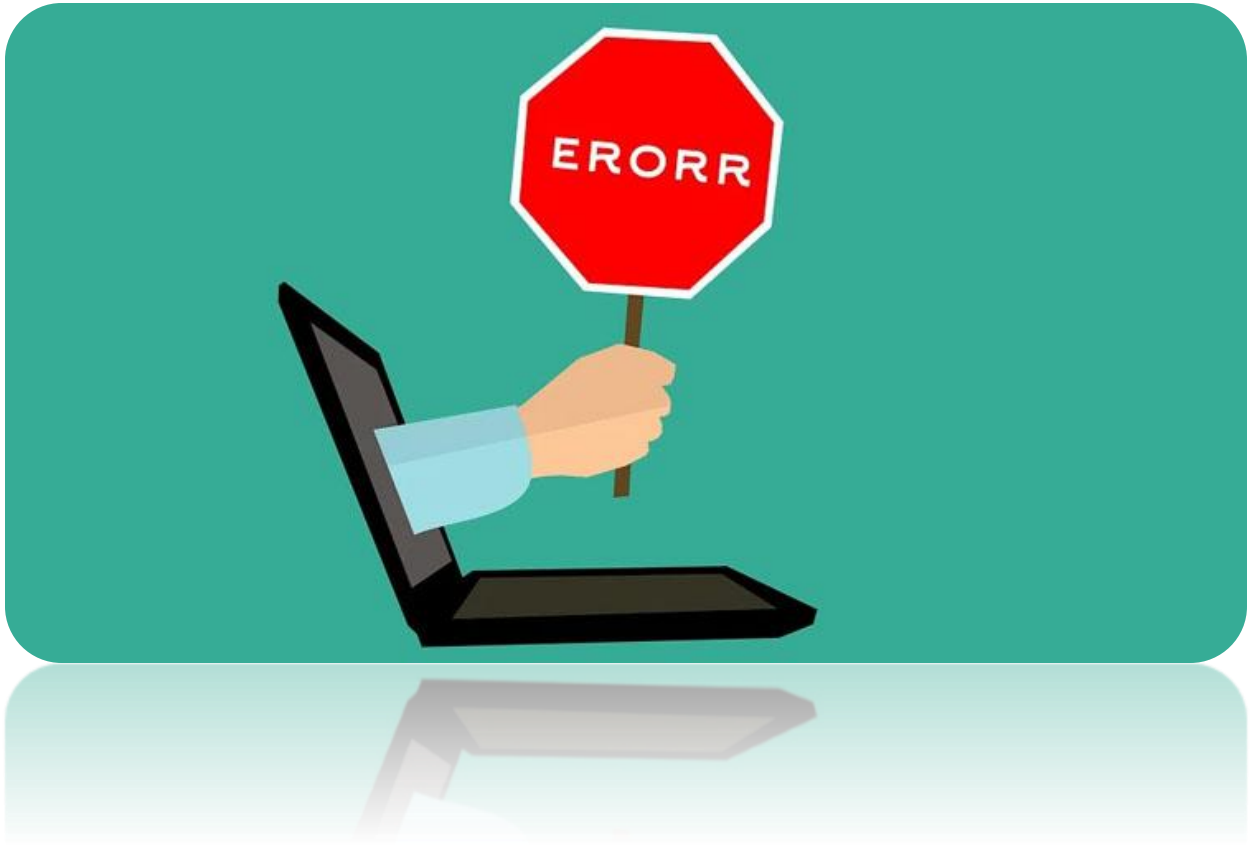
Mostre para alguém e pergunte:

- Você entendeu o que fazer?
- Para onde clicaria?

Se a pessoa se perder → ajuste o wireframe

ERROS COMUNS (evite isso)

- Colocar muito detalhe cedo
- Ignorar mobile
- Layout sem hierarquia
- Botões escondidos
- Falta de espaçamento



DICA DE OURO (nível profissional)
Um bom wireframe responde 3 coisas:

- O que é isso?
- Onde estou?
- O que eu faço agora?

Se não responde isso → precisa melhorar.

9:41
StepStyle
[IMAGEM GRANDE DE TÊNIS]
"Conforto e estilo para o seu dia"
[Botão: Ver produtos]
Destaques
[Imagem]
[Nome]
[Preço]
[Botão]
[Imagem]
[Nome]
[Preço]
[Botão]
[Imagen]
[Nome]
[Preço]
[Botão]
Sobre a loja
"Sobre a loja"
Texto institucional curto | Lorem ipsum dolor sit amet, dia nevismod consectetuer consecttam magna eus eum dolor euismam tincidunt ut laboet dolore nisi ut aliquip ex commodo consequat.
Contato | Redes sociais
© StepStyle 2026

◆ ESTRUTURA DA HOME (copiar no Moqups)

◆ HEADER

[LOGO StepStyle] | [Home | Produtos | Contato]

◆ BANNER (Hero)

[IMAGEM GRANDE DE TÊNIS]

"Conforto e estilo para o seu dia"

[Botão: Ver produtos]

◆ SEÇÃO PRODUTOS

[Título: Destaques]

[Imagem]	[Imagem]	[Imagem]
[Nome]	**[Nome]**	**[Nome]**
[Preço]	[Preço]	[Preço]
[Botão]	**[Botão]**	**[Botão]**

◆ SEÇÃO SOBRE

"Sobre a loja"

Texto institucional curto | Lorem ipsum dolor sit amet, dia nevismod consectetue consectiatum magna eus eum dolor euismam tincidunt ut laboet dolore nisi ut aliquip ex commodo consequat.

◆ RODAPÉ

Contato | Redes sociais

© StepStyle 2026

Semantix

Tarefa no Teams

Senac tecnologia

Sabe o que é uma lista ordenada e uma não ordenada?

Crie um arquivo chamos lista.html depois copie o código ao lado para ver a diferença entre elas.

```html
<!DOCTYPE html>
<html lang="pt-br">
<head>

    <meta charset="UTF-8"><!--Defini Acentuação-->

    <!--Ajusta de acordo com  a tela-->
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Lista Ordenada</title>
</head>
<body>
    <h1>Lista Ordenada</h1>
    <ol>
        <li>Arroz</li>
        <li>Feijão</li>
        <li>Batata</li>
        <li>Tomate</li>
    </ol>
    <hr>
    <h1>Lista não Ordenada</h1>
    <ul>
        <li>Vasco</li>
        <li>Flamengo</li>
        <li>Botafogo</li>
        <li>Fluminense</li>
    </ul>
</body>
</html>
```

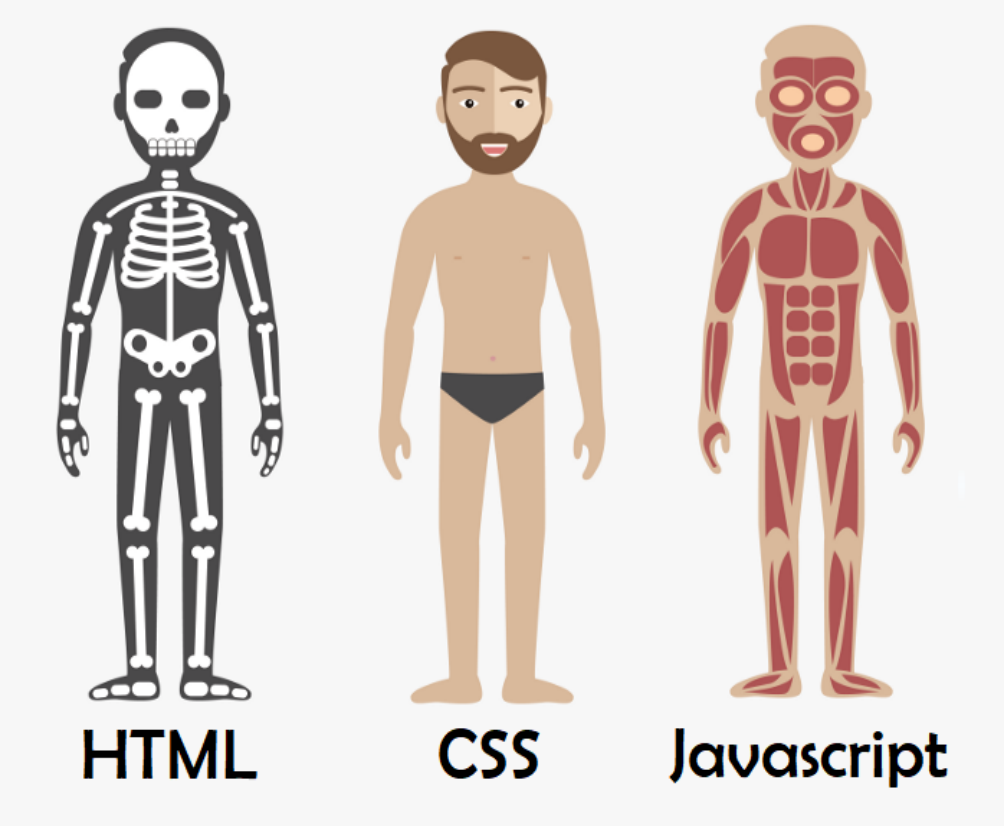
Agora que já entendemos o funcionamento do html que tal começar a maquiagem?

Semantix

CSS – A maquiagem

Senac tecnologia

O HTML é a estrutura
O CSS a maquiagem
O JS os musculos



O que é CSS?

CSS (Cascading Style Sheets) é a linguagem que usamos para estilizar páginas da web. Enquanto o HTML define a **estrutura** do conteúdo, o CSS define a **aparência** (cores, tamanhos, posicionamento, fontes, etc.).

Analogia: HTML é o esqueleto, CSS é a roupa e a maquiagem.

```
:root {
    --white: hsl(4, 0%, 98%);
    --red: hsl(0, 100%, 74%);
    --green: hsl(154, 59%, 51%);
    --blue: hsl(248, 32%, 49%);
    --dark-blue: hsl(249, 10%, 26%);
    --grayish-blue: hsl(246, 25%, 77%);
    --font: 'Poppins', sans-serif;
}
```

Sintaxe Básica

```
seletor {
  propriedade: valor;
  outra-propriedade: outro-valor;
}
```

Exemplo

Isso significa: "Todos os parágrafos (p) terão texto azul e tamanho de fonte 16 pixels".

```
p {
  color: blue;
  font-size: 16px;
}
```

3 formas de usar CSS

CSS Inline (dentro da tag HTML)
CSS Interno (dentro do <head>)
CSS Externo (arquivo separado - melhor prática)

1. CSS Inline (dentro da tag HTML)

Não recomendado para uso geral, útil apenas para testes.

```
<p style="color: red;">Texto vermelho</p>
```

2. CSS Interno (dentro da tag <head>)

```
<style>
  body {
    background-color: lightgray;
  }
</style>
```

3. CSS Externo (arquivo separado – melhor pratica)

```html
<!-- index.html -->
<link rel="stylesheet" href="estilos.css">
```

```css
/* estilos.css */
h1 {
  font-family: Arial;
}
```

Principais seletores

Seletor	Exemplo	O que afeta
Elemento	`p { }`	Todos os parágrafos
Classe	`.destaque { }`	Todos os elementos com `class="destaque"`
ID	`#cabecalho { }`	O elemento com `id="cabecalho"` (único)

Propriedades Essenciais

Propriedade	Valores comuns	Efeito
`color`	`red`, `#FF0000`, `rgb(255,0,0)`	Cor do texto
`background-color`	`lightblue`, `#EEE`	Cor de fundo
`font-size`	`12px`, `1.5rem`, `2em`	Tamanho da fonte
`font-family`	`Arial, sans-serif`	Tipo da fonte
`text-align`	`center`, `right`, `justify`	Alinhamento do texto
`margin`	`10px`, `0 auto`	Espaço externo
`padding`	`5px`	Espaço interno
`border`	`1px solid black`	Borda

Exemplo pratico:

```
.caixa {
  background-color: #f0f0f0;
  padding: 20px;
  border: 1px solid #ccc;
  margin: 10px;
  text-align: center;
}
```

Cores no CSS

- **Nomes:** `red`, `blue`, `green`, `yellow`, `gray`
- **Hexadecimal:** `#FF5733` (RRGGBB)
- **RGB:** `rgb(255, 87, 51)`
- **RGBA** (com transparência): `rgba(255, 87, 51, 0.5)`

Cores no CSS

- **Nomes:** `red`, `blue`, `green`, `yellow`, `gray`
- **Hexadecimal:** `#FF5733` (RRGGBB)
- **RGB:** `rgb(255, 87, 51)`
- **RGBA** (com transparência): `rgba(255, 87, 51, 0.5)`

ATÉ A PRÓXIMA AULA!

Senac tecnologia

Semantix